

SMPK First Pension

Code Handover & Quick Start Guide

Audience: Junior developer / maintenance team

Module: Django backend + React frontend

API prefix: `first-pension/`

Generated from: docs/FIRST_PENSION_*.md

Document index

Part A — Quick Start Checklist

Part B — Full Code Handover

1. What this system does

2. End-to-end workflow

3. Architecture

4. Normal RT vs VR

5. Oracle Forms mapping

6. Database models

7. Backend functions

8. Frontend functions

9. API endpoint index

10. Run & test locally

11. Known gaps

Part A — Quick Start Checklist

Quick Start Checklist — First Pension Module

Use this checklist on Day 1 and when processing a new employee.

Day 1 — Environment setup

- ☐ ☐ Clone repo and open `SMPK/backend/config` + `SMPK/frontend/frontend`
 - ☐ ☐ Backend: activate `venv` → `python manage.py runserver`
 - ☐ ☐ Frontend: `npm install` → `npm run dev`
 - ☐ ☐ Confirm API base URL in `frontend/src/services/Api.js` points to Django
 - ☐ ☐ Read **§1–4** of the full handover doc (what / workflow / VR vs RT)
 - ☐ ☐ Open Oracle `.fmb` reference folder at repo root (optional)
-

Key files — read these first

Priority	File	Why
1	<code>first_pension/urls.py</code>	All API routes
2	<code>first_pension/pension_calculation.py</code>	Pension / gratuity / commutation math
3	<code>first_pension/services/commutation_rate_service.py</code>	Age rate table lookup
4	<code>first_pension/services/pension_bill_service.py</code>	PPN bill + LIC report
5	<code>first_pension/services/sepcom_commutation_generation_service.py</code>	VR separate commutation step 2
6	<code>first_pension/services/ppc_commutation_bill_service.py</code>	VR separate commutation step 3
7	<code>components/EmployeeProcessTabs.jsx</code>	UI workflow shell

Normal retirement (RT) — processing checklist

Employee example: **42230**

- ☐ ☐ **No-Pay tab** — Save no-pay / dies-non days → `POST no-pay/`
- ☐ ☐ **Proposal tab** — Save proposal (separation RT, bank, CA) → `POST pension-proposal/`

- [] **Commutation tab** — Save application with **PEN** (with pension) → POST `commutation/`
- [] **Amount tab** — Calculate → POST `amount/calculate/` (check pension + commutation + gratuity)
- [] **Amount tab** — First-month generate → POST `amount/first-month-generate/`
- [] **Bill** → **PPN** — Generate bill for correct month/year
- [] **Bill** → **Journal Voucher** — Generate JV for PPN bill
- [] **Reports** — LIC report, Pension Sanction, Bill Abstract, Journal Summary

Verify: `PensionSummary.pension_amount` = Oracle `PEN_AMT`; commutation lump sum uses age from `fi_pn_md_commrte_rupee`.

Voluntary retirement (VR) — two-phase checklist

Employee example: **43186**

Phase A — First pension (no commutation)

- [] Proposal: separation type **VR**
- [] Commutation application can wait OR save early with **COM** for later
- [] Amount: pension + gratuity only (commutation shows **Deferred**)
- [] Bill → **PPN only** (no commutation line)
- [] Reports: LIC + Pension Sanction

Phase B — Separate commutation (later)

- [] **Commutation tab** — Application saved, **COM** (Separate Commutation), bank, dates, comm %
- [] **Commutation tab** — **Generate SEPCOM** for correct month/year (e.g. 01/2024)
- [] **Bill** → **PPC** — Use **same month/year as SEPCOM** (not current year)
- [] **Reports** → **Commutation Sanction** — Uses `sep-comm-report` after SEPCOM

Watch out: SEPCOM amount must use **disbursed pension** (Oracle pensioner), not a recalculated higher summary — see Known Gaps in full doc.

Debug cheat sheet

Symptom	Check
Amount won't calculate	No-pay saved? Commutation % saved? (not required for VR)
Commutation report fails	SEPCOM generated? Backend error log (earn/dedn field names)
PPC shows no candidates	Bill month/year must match SEPCOM (e.g. 1/2024 not 2026)
Wrong commutation lump sum	Compare <code>PensionSummary.pension_amount</code> vs Oracle <code>PEN_AMT</code>
Wrong age rate	DOB + application received / MO cert date → <code>fi_pn_md_commrte_rupee</code>

Hub API: GET `first-pension/employees/<emp_id>/` — shows which tabs are complete.

API quick index (most used)

GET	employees/<id>/	Workflow hub
POST	no-pay/	No-pay save
POST	pension-proposal/	Proposal save
POST	commutation/	Commutation application
GET	amount/employee/<id>/	Amount tab load
POST	amount/calculate/	Run calculation
POST	amount/first-month-generate/	Post FMPEN
POST	pension-bill/generate/	PPN bill
GET	pension-bill/sepcom/status/	SEPCOM status
POST	pension-bill/sepcom/generate/	SEPCOM create
POST	pension-bill/ppc/generate/	PPC bill
GET	pension-bill/sep-comm-report/	VR commutation report
GET	pension-bill/commutation-report/	RT commutation report
GET	pension-bill/lic-report/	LIC report

Oracle form → code map (memorize)

Step	Oracle	SMPK service / UI
Application	FI_PN_MH_APPLICATION	CommutationApplicationEntry
Amount / FMPEN	FI_PN_FIRST_PENSION_PROCESS	pension_calculation + first_month_pension_service
SEPCOM	FI_PN_COMMUTATION_GENERATION	sepcom_commutation_generation_service
PPC bill	FI_PN_TH_Commutation_Bill_Gen	ppc_commutation_bill_service
Sep report	SEP_COMM_REP1	sep_comm_report_service
PPN bill	Bill generate FMB	pension_bill_service

Full function-by-function reference: see *FIRST_PENSION_CODE_HANDOVER.md*

Part B — Full Code Handover

SMPK First Pension — Code Handover Document

Audience: Junior developer taking over maintenance and enhancements

Module: First Pension processing (Django backend + React frontend)

Last updated: June 2026

Backend root: SMPK/backend/config/first_pension/

Frontend root: SMPK/frontend/frontend/src/components/ (pension UI)

API prefix: first-pension/ (mounted in Django config/api/urls.py)

Table of contents

1. What this system does
 2. End-to-end business workflow
 3. Architecture overview
 4. Normal retirement vs VR (Voluntary Retirement)
 5. Oracle Forms mapping
 6. Database models (business meaning)
 7. Backend — function reference by file
 8. Frontend — function reference by file
 9. Complete API endpoint index
 10. How to run and test locally
 11. Known gaps and pending work
-

1. What this system does

SMPK First Pension is a port of the Oracle Forms pension workflow used at Syama Prasad Mookerjee Port, Kolkata. It lets finance/pension staff:

1. Record employee separation and service adjustments (no-pay, dies-non)
2. Enter pension proposal and commutation application
3. Calculate pension, commutation lump sum, and gratuity
4. Generate first-month pension (FMPEN) and bank bills (PPN)
5. For **separate commutation** (mainly VR): generate SEPCOM → PPC bill → reports
6. Print sanction reports, LIC reports, bill abstracts, and journal summaries

Design principle: Business rules follow Oracle Forms (.fmb) behaviour. MySQL tables prefixed `fi_pn_*` mirror Oracle `FINANCE.FI_PN_*` tables.

2. End-to-end business workflow

UI tab order (EmployeeProcessTabs.jsx)

Step	Tab	Business step	Must complete before
1	Basic Info	Read-only employee snapshot	—
2	Commutation	Commutation application (+ SEPCOM for COM)	Proposal recommended
3	No-Pay	Service adjustment days	Employee selected
4	Proposal	Pension proposal header + earn/dedn	No-pay saved
5	Amount	Calculate pension / gratuity / commutation	Proposal + commutation %
6	Bill & Journal	PPN bill, PPC bill, JV	Amount + first-month generate
7	Reports	LIC, sanction, commutation, abstract, JV summary	Relevant bill generated

Typical path — normal retirement (RT)

No-Pay → Proposal → Commutation (PEN) → Amount → First-month generate → PPN Bill → JV → Reports

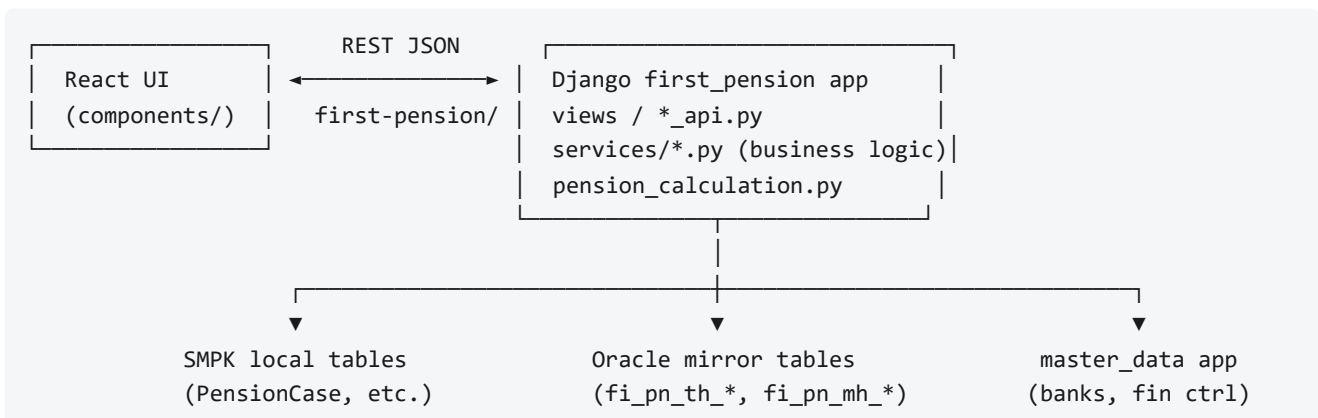
- Commutation is **with first pension** (`impl_fpen_combill = PEN`).
- Lump sum calculated on Amount tab using age rate table.

Typical path — voluntary retirement (VR)

No-Pay → Proposal (VR) → Amount (pension + gratuity only) → PPN Bill → Reports
↓ (later, when employee applies)
Commutation (COM) → SEPCOM → PPC Bill → Commutation Sanction report

- First bill has **no commutation line**.
- Separate commutation uses 4 Oracle steps (application → SEPCOM → PPC → report).

3. Architecture overview



Layer	Responsibility
<code>*_api.py / views.py</code>	HTTP validation, auth, audit logging, call services
<code>services/*.py</code>	Oracle-equivalent business processes (bills, SEPCOM, reports)
<code>pension_calculation.py</code>	Pure financial formulas (TCCS, TQS, pension, gratuity)
<code>oracle_mirror.py</code>	Django models → existing MySQL Oracle mirror tables
<code>models.py</code>	SMPK workflow tables (case, proposal, commutation application)
React components	One tab or sub-workflow per component; call API via <code>services/Api.js</code>

4. Normal retirement vs VR (Voluntary Retirement)

Topic	Normal (RT)	VR
Separation type	RT	VR
Commutation timing	With first pension (PEN)	Deferred (COM = separate)
Amount tab commutation	Calculated and saved	Shows Deferred ; amount = 0
SEPCOM / PPC	Not used	Required for commutation payment
Pension for commutation lump sum	Same as Amount tab summary	Should use disbursed pension from pensioner/application (Oracle); SMPK currently uses summary — see §11
Age rate table	<code>fi_pn_md_commrates_rupee</code>	Same table
Age reference date	MO cert date if application > 12 months after separation; else application received date	Same rule

Key functions: `is_voluntary_retirement()`, `build_amount_lookup_payload()` in `pension_calculation.py`.

5. Oracle Forms mapping

Oracle Form	SMPK implementation
FI_PN_MH_APPLICATION.fmb	CommutationApplicationEntry.jsx + CommutationCreateAPIView
FI_PN_FIRST_PENSION_PROCESS.fmb	pension_calculation.py + first_month_pension_service.py
FI_PN_COMUTATION_GENERATION.fmb	sepcom_commutation_generation_service.py + PensionSepcomGeneration.jsx
FI_PN_TH_Commutation_Bill_Gen.fmb	ppc_commutation_bill_service.py + PensionPpcBillGeneration.jsx
SEP_COMM_REP1.fmb	sep_comm_report_service.py
First pension bill generation	pension_bill_service.generate_pension_bills
LIC report	pension_bill_service.build_lic_report
JV generation	voucher_generation_service.py

6. Database models (business meaning)

SMPK local (models.py)

Model	Business purpose
PensionCase	Employee snapshot + no-pay inputs + separation; one row per employee
PensionSummary	Calculated service (TCCS/TQS) and amounts after Amount tab calculate
CommutationApplication	Commutation application form data (one per employee)
PensionProposal	Pension proposal header (CA number, bank, separation, holds)
PensionProposalEarndedn	Proposal earn/deduction grid lines

Oracle mirrors (oracle_mirror.py)

Model	Oracle table	Business purpose
FiPnMhApplication	FI_PN_MH_APPLICATION	Commutation application mirror
FiPnMhPensionProposal	FI_PN_MH_PENSION_PROPOSAL	Proposal header mirror
FiPnMhPensioner	FI_PN_MH_PENSIONER	Active pensioner (pension, commuted portion)
FiPnThFirstMonthPension	FI_PN_TH_FIRST_MONTH_PENSION	First-month bill header (FMPEN_ID)
FiPnTdFirstMonthPension	FI_PN_TD_FIRST_MONTH_PENSION	First-month earn/dedn lines
FiPnThPensionBill	FI_PN_TH_PENSION_BILL	Bank bill header (PPN or PPC)
FiPnThSepcom / FiPnTdSepcom	FI_PN_TH/TD_SEPCOM	Separate commutation generation
FiPnMdCommrateRupee	FI_PN_MD_COMMRATE_RUPEE	Commutation rate by age
FiPnThJv / FiPnTdJv	Journal voucher header/detail	
FiPnMhPmthsetup	Month open/close control per bill type	

7. Backend — function reference by file

Convention: Each entry = **Function** → **Business logic** → **Called from**

urls.py

Route definitions only. All paths under first-pension/. See [§9 API index](#).

Function / Class	Business logic	Called from
<code>_format_date_for_api</code>	Formats date for JSON (YYYY-MM-DD)	Serializers
<code>_parse_optional_date</code>	Parses optional date from request	Commutation save
<code>_apply_commutation_fields</code>	Maps POST body → <code>CommutationApplication</code> ; sets <code>restoration = commutation + 15 years</code> ; VR defaults <code>impl_fpen_combill=COM</code>	Commutation API
<code>serialize_no_pay_data</code>	Builds no-pay GET response	NoPayLookupAPIView
<code>_get_pension_case_for_employee</code>	Finds <code>PensionCase</code> by emp code variants	Multiple views
<code>_parse_dd_mm_yyyy</code>	Parses DD-MM-YYYY dates	Legacy process
<code>serialize_commutation_application</code>	Full commutation form JSON	Commutation API
<code>_enrich_commutation_data_from_proposal</code>	Fills CA/bank from proposal if missing	Employee search
PensionProcessView	Legacy: one-shot create case + inline calculation (prefer Amount tab flow)	POST <code>process/</code>
PensionReportView	Legacy single-case report	GET <code>report/<id>/</code>
PensionCaseListView	Lists all pension cases	GET <code>cases/</code>
<code>_build_employee_db_context</code>	Assembles flags: intake, no-pay, proposal, commutation, amount done	Employee search
EmployeeSearchAPIView	Hub: Oracle employee + all tab completion state	GET <code>employees/<id>/</code>
NoPayLookupAPIView	Load no-pay days for employee	GET <code>no-pay/employee/</code>
NoPayEntryAPIView	Create/update <code>PensionCase</code> with service adjustments	POST/PUT <code>no-pay/</code>
CommutationCreateAPIView	Create/update commutation application; allocate <code>appcn_no</code>	POST/PUT <code>commutation/</code>
<code>_sync_commutation_to_oracle</code>	Push to Oracle mirror (may be disabled)	Commutation save

pension_calculation.py

Function	Business logic	Notes
PensionCalculationError	Exception for calculation failures	
round_up_to_rupee	ceil() — used for SEPCOM lump sum	Oracle Ceil
highest_side_round	Floor +1 if fraction > 0.009	Oracle FFUNC_HIGHEST_SIDE_ROUND; used for integrated commutation
calculate_normal_pension	Government: 50% of emoluments; Port: emoluments × min(TQS,30) / 80; floor Rs 1850	Port of FFUNC_NORMAL_PENSION
get_pension_option_for_employee	Reads G/P from proposal	
get_commutation_application_for_employee	Finds CommutationApplication	
get_commutation_percent_for_employee	Reads commutation %; errors if not saved when required	
reload_pension_case_from_db	Fresh PensionCase + related data	Always use before calculate
get_separation_type_for_employee	RT / VR / etc. from proposal or case	
is_voluntary_retirement	True if separation type = VR	Defers commutation
fetch_calculation_inputs_from_db	Merges case, commutation %, Oracle oldbill overrides for no-pay	
calculate_pension_financials	Core math: TCCS/TQS from join/retire dates; pension; commutation via rate table; gratuity (cap 20 lakh)	Pure function
serialize_amount_data	Amount tab display structure	
run_pension_calculation_for_case	Runs calculate + saves PensionSummary	Amount Calculate API
format_total_service / format_service_tenure_years_only	Display helpers	Proposal UI
get_service_components_for_case	Service Y/M/D for display	
build_amount_lookup_payload	GET amount tab: inputs + summary + VR flags + first-	Amount Lookup API

Function	Business logic	Notes
	month status	

Commutation formula (integrated):

```
monthly_commuted = trunc(pension × comm% / 100)
lump_sum = highest_side_round(monthly_commuted × rate_per_rupee)
```

Gratuity: $(\text{basic} + \text{DA}) \times 15 \times \text{qualifying_years} / 26$, cap Rs 20,00,000.

pension_amount_api.py

Class	Business logic
PensionAmountLookupAPIView	GET fresh amount tab state via build_amount_lookup_payload
PensionAmountCalculateAPIView	POST → run_pension_calculation_for_case + audit
FirstMonthPensionGenerateAPIView	POST → generate_first_month_pension (creates FMPEN + pensioner)

pension_bill_api.py

Thin DRF wrappers. Each POST logs audit via `audit.services.log_audit`.

Class	Delegates to	Business step
PensionBillStatusAPIView	get_employee_bill_status	Employee bill pipeline state
PensionBillCandidatesAPIView	list_bill_candidates	Who can be billed this month
PensionBillGenerateAPIView	generate_pension_bills	Create PPN bills
PensionBillMonthStatusAPIView	get_month_setup_status	Month open/closed
PensionBillReprocessCandidatesAPIView	list_reprocess_candidates	Bills needing regeneration
PensionBillReprocessAPIView	reprocess_pension_bills	Regenerate bill totals
PensionBillCloseMonthAPIView	close_pension_bill_month	Close pension bill month
PensionBillPpnListAPIView	list_ppn_bills	PPN list for LOV
PensionLicReportAPIView	build_lic_report	LIC treasurer report
PensionBillAbstractReportAPIView	build_bill_abstract_report	Payment abstract
PensionJournalSummaryReportAPIView	build_journal_summary_report	JV summary print
PensionJournalSummaryBillsAPIView	list_bills_for_journal_report	Bills for JV LOV
PensionProposalSanctionReportAPIView	build_proposal_sanction_report	Pension sanction letter
PensionSepcomStatusAPIView	get_sepcom_status	SEPCOM readiness
PensionSepcomGenerateAPIView	generate_sepcom_for_employees	Create SEPCOM rows
PensionSepCommReportAPIView	build_sep_comm_report	Separate commutation report
PensionPpcBillStatusAPIView	get_ppc_employee_status	PPC pipeline state
PensionPpcBillCandidatesAPIView	list_ppc_candidates	SEPCOM rows ready for PPC
PensionPpcBillGenerateAPIView	generate_ppc_bills	Create PPC bills
PensionPpcBillListAPIView	list_ppc_bills	PPC bills for month
PensionCommutationBillReportAPIView	build_commutation_bill_report	Commutation sanction (PEN path)
PensionVoucherStatusAPIView	get_voucher_status	JV exists for bill?
PensionVoucherPreviewAPIView	preview_ppn_voucher	Dry-run JV lines
PensionVoucherGenerateAPIView	generate_ppn_voucher	Post JV
PensionManualJournalTemplateAPIView	blank_manual_journal_template	Empty manual JV form
PensionManualJournalSummaryAPIView	get_journal_summary	JV summary lookup
PensionManualJournalListAPIView	list_journals	List JVs for period

Class	Delegates to	Business step
PensionManualJournalSaveAPIView	save_manual_journal	Save manual JV with balance check

services/commutation_rate_service.py

Function	Business logic
CommutationRateError	Missing DOB, age, or rate in master
age_next_birthday	Oracle FFUNC_AGE_NEXT_BIRTHDAY — age for rate table lookup
resolve_age_reference_date	If application received >12 months after separation → MO certification date; else application received date
get_rate_per_rupee	Reads fi_pn_md_commrate_rupee by AGE_YRS, latest WEF_DT ≤ reference date
resolve_commutation_context	Collects DOB, dates, computed age for an employee
compute_commutation_amounts	$\text{trunc}(\text{pension} \times \text{comm} \% / 100) \times \text{rate} \times \text{share} \% / 100$, rounded
compute_commutation_for_employee	End-to-end commutation math for one emp_cd

services/commutation_defaults_service.py

Function	Business logic
load_commutation_defaults	Prefill commutation form from Oracle FiPnMhApplication; sets VR flag and default COM

services/sepcom_commutation_generation_service.py

Oracle step 2 — FI_PN_COMUTATION_GENERATION.fmb

Function	Business logic
SepcomGenerationError	SEPCOM-specific errors
_format_sepcom_id	SEP/MM/YYYY/serial
_allocate_sepcom_serial	Next serial from fin control or max+1
_get_comm_app	Find commutation application
_commutation_period	Bill month/year from commutation or application date
_resolve_bank_cd	Bank from commutation app or proposal
calculate_commutation_amount	Lump sum from pension summary + commutation app + age rate
get_sepcom_status	Readiness flags for UI
_can_generate_sepcom	Validates COM mode, no duplicate SEPCOM, amounts, bank
_sync_mirror_commutation_amt	Updates Oracle application mirror commutation amount
_update_pensioner_commuted_portion	Sets commuted_portion on pensioner master
generate_sepcom_for_employees	Creates FiPnThSepcom + FiPnTdSepcom (earn code 203)

services/ppc_commutation_bill_service.py

Oracle step 3 — FI_PN_TH_Commutation_Bill_Gen.fmb

Function	Business logic
_format_ppc_bill_no	PPC/MM/YYYY/serial
_allocate_ppc_serial	Serial from fin control
_ensure_month_setup	Creates month setup row for bill type C
_sum_sepcom_earn_dedn	Sums SEPCOM TD lines by earn/dedn type
_validate_ppc_candidate	SEPCOM exists, not already billed, bank present, amounts > 0
serialize_ppc_candidate	One row for PPC candidate grid
list_ppc_candidates	Eligible SEPCOM headers for month
get_ppc_employee_status	Per-employee PPC state for UI
generate_ppc_bills	Groups by bank; creates FiPnThPensionBill type C; links SEPCOM + application bill_no
list_ppc_bills	Lists PPC bills for month

services/sep_comm_report_service.py

Oracle step 4 — SEP_COMM_REP1.fmb

Function	Business logic
_build_sepcom_page	One report page from SEPCOM header + TD lines
build_sep_comm_report	Full report payload; enriches with sanction narrative from commutation report

services/commutation_bill_report_service.py

Function	Business logic
_build_row	One employee sanction letter row (pension, MO cert, age narrative, lump sum)
build_commutation_bill_report	Recommendation & Sanction of Commutation (used for PEN and as fallback)

services/first_month_pension_service.py

Function	Business logic
get_first_month_status	Whether FMPEN already posted for employee
generate_first_month_pension	Allocates FMPEN_ID + PPN bill no; posts header/detail; creates/updates pensioner; skips commutation line for VR

`services/pension_bill_service.py`

Function	Business logic
<code>please_sum_pension</code>	Sums pension amounts for bill type/month
<code>list_bill_candidates</code>	FMPEN rows ready for PPN billing
<code>generate_pension_bills</code>	Oracle FPROC_BILL_GENERATE — group by bank, allocate PPN numbers
<code>get_employee_bill_status</code>	Employee position in PPN pipeline
<code>build_lic_report</code>	LIC treasurer report; ID card holdup line when not submitted
<code>list_reprocess_candidates / reprocess_pension_bills</code>	Regenerate when source data changed
<code>close_pension_bill_month</code>	Close month in <code>FiPnMhPmthsetup</code>
<code>repair_orphan_pension_bill</code>	CLI repair for broken bill links

`services/commutation_appcn_service.py` *(if present in repo)*

Function	Business logic
<code>get_next_appcn_no_with_fallback</code>	Next application number from Oracle or local max+1
<code>resolve_appcn_no_for_create</code>	Uses submitted or auto-allocated appcn_no

`pension_proposal_api.py` *(referenced in `urls.py`)*

Class / Function	Business logic
<code>PensionProposalLookupAPIView</code>	Load proposal or Oracle defaults
<code>PensionProposalAPIView</code>	Save proposal + earn/dedn grid
<code>validate_pension_proposal_data</code>	Oracle-equivalent validation rules
Bank / EarnDedn API views	Master data LOVs for proposal form

`process_intake_api.py` (*referenced in `urls.py`*)

Class / Function	Business logic
<code>ProcessIntakeLookupAPIView</code>	Separation intake status
<code>ProcessIntakeAPIView</code>	Save separation type/date on case + proposal

`utils/amount_words.py`

Function	Business logic
<code>rupees_amount_in_words</code>	Indian numbering — amount in words for report footers

8. Frontend — function reference by file

Note: Full UI references additional components (`NoPayEntry`, `PensionProposalEntry`, `PensionReports`, etc.) imported by `EmployeeProcessTabs.jsx`. Only files currently under `src/components/` are listed in detail below.

`EmployeeProcessTabs.jsx`

Function	Business logic
<code>formatAge</code>	Display {years, months, days}
<code>INFO_FIELDS</code>	Basic Info grid field definitions
<code>EmployeeProcessTabs</code>	Renders 7 workflow tabs and child components

Tabs rendered: Basic Info → Commutation → No-Pay → Proposal → Amount → Bill & Journal → Reports

CommutationApplicationEntry.jsx

Function	Business logic
toInputDate	Normalize dates for <code><input type="date"></code>
restorationFromCommutation	Restoration date = commutation date + 15 years
resolveBankFromMaster	Bank code → description
isVoluntaryRetirement	Detect VR → default Separate Commutation
buildNewCommutationForm	New form defaults from employee/proposal
mapCommutationDataToForm	Saved record → form state
CommutationApplicationEntry	Main form: load/save commutation application

API calls:

- GET banks/, GET banks/<code>/
- GET commutation/ (next appcn_no)
- POST commutation/, PUT commutation/<id>/

Key fields: impl_fpen_combill (PEN vs COM), commutation_per, dates, bank, ref_no, bill_no, MO certificate.

PensionSepcomGeneration.jsx

Function	Business logic
formatMoney	Indian number format
loadStatus	GET SEPCOM status; sync bill month/year from response
handleGenerate	POST SEPCOM generation for employee
PensionSepcomGeneration	Step 2 separate commutation UI

API: GET sepcom/status/employee/, POST sepcom/generate/

Prerequisites: Commutation application saved with COM; amount calculated; bank present.

PensionAmountEntry.jsx

Function	Business logic
formatMoney	Display formatting
loadFromDb	GET amount tab state
handleCalculate	POST calculate → saves summary
handleFirstPensionGenerate	POST first-month generate (FMPEN)
PensionAmountEntry	Amount tab UI

API: GET amount/employee/, POST amount/calculate/, POST amount/first-month-generate/

UI rules: Blocks calculate if no-pay missing; shows VR deferred commutation message.

PensionBillJournalTabs.jsx

Function	Business logic
PensionBillJournalTabs	Sub-tabs: PPN Bill, PPC Bill, Journal Voucher, Manual Journal
useEmployeeBillContext	Hook: shared bill no / JV period (from hooks/useEmployeeBillContext.js)
refreshBillContext	Reload on tab show

Child components: PensionBillGeneration, PensionPpcBillGeneration, PensionVoucherGeneration, PensionManualJournal

PensionPpcBillGeneration.jsx

Function	Business logic
loadData	PPC status + month close + candidates; auto-set month/year from sepcom_month/year
toggleRow	Select employees for batch PPC
handleGenerate	POST PPC bill generation
PensionPpcBillGeneration	Separate commutation bill UI

API: GET ppc/status/, GET month-status/, GET ppc/candidates/, POST ppc/generate/

Important: Use SEPCOM month/year (e.g. 01/2024), not current calendar year.

`PensionCommutationBillReport.jsx`

Function	Business logic
<code>loadReport</code>	Tries <code>sep-comm-report</code> first; falls back to <code>commutation-report</code>
<code>handlePrint</code>	Browser print
<code>PensionCommutationBillReport</code>	Commutation Sanction report tab

`PensionLicReportPrint.jsx`

Function	Business logic
<code>LicReportPage</code>	Renders one LIC report page (presentational)
<code>PensionLicReportPrint</code>	Print layout for LIC report

No API calls — parent loads data and passes `report` prop.

Expected components (imported but may live outside current `src/` snapshot)

Component	Tab	Purpose
<code>NoPayEntry</code>	No-Pay	Save no-pay / dies-non days
<code>PensionProposalEntry</code>	Proposal	Pension proposal form
<code>PensionBillGeneration</code>	Bill → PPN	First pension bill generation
<code>PensionVoucherGeneration</code>	Bill → JV	Auto journal voucher
<code>PensionManualJournal</code>	Bill → Manual JV	Manual journal entry
<code>PensionReports</code>	Reports	Container for all report sub-tabs
<code>PensionBillContextBar</code>	Bill / Reports	Shows bill no, JV period, refresh
<code>PensionCommutationBillPrint</code>	Reports	Print layout for commutation sanction
<code>services/Api.js</code>	All	Axios wrapper for backend
<code>hooks/useEmployeeBillContext.js</code>	Bill	Bill status hook

9. Complete API endpoint index

Method	Path	Purpose
GET	employees/<emp_id>/	Employee hub + workflow flags
GET/POST	process-intake/...	Separation intake
GET/POST/PUT	no-pay/...	No-pay entry
GET/POST/PUT	pension-proposal/...	Proposal
GET/POST/PUT	commutation/...	Commutation application
GET	amount/employee/<code>/	Amount tab load
POST	amount/calculate/	Run calculation
POST	amount/first-month-generate/	Post FMPEN
GET	pension-bill/status/employee/<code>/	PPN bill status
GET	pension-bill/candidates/	PPN candidates
POST	pension-bill/generate/	Generate PPN
GET	pension-bill/sepcom/status/employee/<code>/	SEPCOM status
POST	pension-bill/sepcom/generate/	Generate SEPCOM
GET	pension-bill/ppc/status/employee/<code>/	PPC status
GET	pension-bill/ppc/candidates/	PPC candidates
POST	pension-bill/ppc/generate/	Generate PPC
GET	pension-bill/lic-report/	LIC report
GET	pension-bill/commutation-report/	Commutation sanction
GET	pension-bill/sep-comm-report/	Separate commutation report
GET	pension-bill/proposal-sanction-report/	Pension sanction
GET	pension-bill/bill-abstract-report/	Bill abstract
GET	pension-bill/journal-summary-report/	JV summary
GET/POST	pension-bill/voucher/...	Auto JV
GET/POST	pension-bill/manual-journal/...	Manual JV

Full route list: `first_pension/urls.py`.

10. How to run and test locally

Backend

```
cd SMPK/backend/config
../venv/Scripts/python.exe manage.py runserver
```

Frontend

```
cd SMPK/frontend/frontend
npm install
npm run dev
```

Suggested test employees

emp_cd	Type	Use for
42230	Normal RT	Full PEN flow; amounts should match Oracle
43186	VR	Separate COM flow; SEPCOM → PPC → report

Debug tips

1. Always check `GET first-pension/employees/<id>/` for workflow flags first.
2. Amount issues → trace `run_pension_calculation_for_case` → `calculate_pension_financials`.
3. SEPCOM amount → trace `calculate_commutation_amount` → `commutation_rate_service`.
4. Report failures → check Django console for `FieldError` or missing prerequisites.
5. Oracle mirror tables (`fi_pn_*`) are read/write — compare with SMPK local tables.

11. Known gaps and pending work

Item	Description
VR SEPCOM pension source	SEPCOM uses <code>PensionSummary.pension_amount</code> ; Oracle uses disbursed pension from pensioner/application — can mismatch (see emp 43186)
Oracle sync on save	<code>_sync_*_to_oracle</code> functions may be disabled — local save works; mirror may lag
Frontend completeness	Some tab components referenced but not in minimal <code>src/</code> snapshot — verify full deployment tree
SEPCOM delete/regenerate	Oracle <code>FPROC_DELETE</code> not yet ported
Nominee split	Multiple SEPCOM rows per employee not implemented
PPC JV	Journal voucher for PPC bills (parallel to PPN PNJV)

Quick reference — who calls whom

Amount Calculate

- `run_pension_calculation_for_case`
 - `calculate_pension_financials`
 - `commutation_rate_service.compute_commutation_amounts`

First-month Generate

- `generate_first_month_pension`
 - `FiPnThFirstMonthPension`, `FiPnMhPensioner`

PPN Bill Generate

- `generate_pension_bills`
 - `FiPnThPensionBill` (type N)

SEPCOM Generate

- `generate_sepcom_for_employees`
 - `calculate_commutation_amount`
 - `FiPnThSepcom`, `FiPnTdSepcom`

PPC Bill Generate

- `generate_ppc_bills`
 - `FiPnThPensionBill` (type C)

Commutation Report

- `build_sep_comm_report` (COM) OR `build_commutation_bill_report` (PEN)

Document maintained for SMPK First Pension handover. For questions about Oracle business rules, refer to `.fmb` files in repo root (e.g. `FI_PN_COMUTATION_GENERATION.fmb`).